



# SOVEREIGN OS DIM

## Guide Développeur

---

Version 37.5 - 8 sections - référence technique

GHT Psy Sud Paris - Fondation Vallée + Paul Guiraud

*Édition V37.5 (2026)*

*Adam Beloucif - [adam.beloucif@psysudparis.fr](mailto:adam.beloucif@psysudparis.fr)*

Ce guide est destiné aux développeurs, aux équipes DSI et aux contributeurs du projet. Il documente l'architecture, la stack, le pont local in-process, le module ML, la sécurité et la CI/CD. Pour le guide d'utilisation quotidienne (TIM, médecin DIM, chef de pôle), voir Sovereign\_OS\_DIM\_Guide.pdf.

**Dépôt - [https://github.com/Adam-Blf/sovereign\\_os\\_dim](https://github.com/Adam-Blf/sovereign_os_dim) - Stack - Python 3.12 + C# .NET 8 + WebView2 + XGBoost**

# Sommaire

---

1. **Architecture générale**
2. **Stack technique et dépendances**
3. **Pont local in-process (pywebview)**
4. **Module ML - configuration et entraînement**
5. **Sécurité et conformité RGPD**
6. **CI/CD - pipeline et qualité de code**
7. **Performances et benchmarks**
8. **Contribuer au projet**

## Architecture générale

### Vue d'ensemble

Sovereign OS DIM est un monolithe desktop PyInstaller, 100% local. L'hôte (pywebview en Python, ou C# .NET 8 + WebView2 pour le portage) affiche un frontend HTML/CSS/JS dans une fenêtre native. Le backend Python s'exécute dans le même processus et n'ouvre aucun serveur : le frontend l'appelle via le pont in-process de pywebview (js\_api). Aucune socket, aucun port, aucun flux réseau.

### Couches

| Couche       | Technologie                    | Rôle                           |
|--------------|--------------------------------|--------------------------------|
| Hôte desktop | pywebview (ou C# .NET 8 + WebV | Fenêtre native, pont js_api, p |
| Frontend     | HTML + Tailwind + Chart.js (ve | UI rendue dans WebView2        |
| Pont local   | backend/interfaces/api.py + _s | Appels in-process, aucun serve |
| Données      | SQLite locale                  | MPI persisté, résolutions IDV  |
| ML           | XGBoost, LightGBM, scikit-lear | Assistance TIM (3 modèles)     |
| PDF          | fpdf2 >= 2.8 (LGPL) + Montserr | Génération guides + exports    |

### Flux de données

1. L'utilisateur interagit avec le frontend (WebView2).
2. Le JavaScript appelle `window.pywebview.api.<methode>()` : l'appel traverse le pont natif, sans HTTP ni socket.
3. Le backend Python traite les fichiers ATIH en parallèle (ThreadPoolExecutor, max 8 workers) et écrit en SQLite.
4. Les résultats reviennent en dictionnaires JSON-sérialisables.

Aucune couche réseau ne doit être réintroduite : le test `tests/test_no_network.py` échoue si un serveur HTTP (Flask, FastAPI, uvicorn) réapparaît dans l'application.

## Stack technique et dépendances

### Backend Python

| Package      | Version | Usage                          |
|--------------|---------|--------------------------------|
| Python       | 3.12    | Runtime principal              |
| fpdf2        | >= 2.8  | Génération PDF (LGPL)          |
| xgboost      | >= 2.0  | format_detector + collision_ri |
| lightgbm     | >= 4.0  | Benchmark ML (challenger)      |
| scikit-learn | >= 1.4  | ddn_validity (RandomForest) +  |
| pandas       | >= 2.0  | Chargement dataset synthétique |
| pypdf        | >= 4.0  | Enrichissement bookmarks PDF   |
| PyInstaller  | >= 6.0  | Packaging .exe portable        |

### Frontend

HTML/CSS vanille + Tailwind CSS et Chart.js vendorisés en local (frontend/vendor, aucun CDN). Chart.js pour les graphiques. Aucun framework JS (React/Vue) - volontairement léger pour rester embarquable dans WebView2 sans build step. ES modules natifs (import/export).

### C# Desktop Shell

Portage optionnel de l'hôte : Microsoft.Web.WebView2 pour l'affichage HTML, Microsoft.Data.Sqlite pour l'accès au MPI, WinForms comme conteneur de fenêtre. Le shell C# charge le même frontend et expose un pont natif équivalent au js\_api de pywebview, toujours sans serveur ni socket.

### Installation de l'environnement de développement

```
# Python (backend + ML + PDF)
pip install -r requirements.txt

# Tests
pytest tests/ -v

# Lint + sécurité
ruff check .
bandit -r backend/
python -m pytest tests/test_no_network.py

# Build .exe
python build.py
```

## Pont local in-process (pywebview)

### Principe

Le frontend n'utilise aucun serveur : chaque méthode publique de la classe `Api` (`backend/interfaces/api.py`) est exposée au JavaScript par `pywebview` sous `window.pywebview.api.<methode>()`. La logique des écrans Sentinel v2 (`cockpit`, `ML`, `audit`, `CeSPA`, `diff`, `heatmap`, `twin`, `workflow`) vit dans `backend/interfaces/_sentinel.py` et partage le même `DataProcessor` que le reste de l'application.

```
# Côté JavaScript (frontend/js/sentinel-helpers.js)
const r = await window.pywebview.api.get_cockpit();

# Côté Python (backend/interfaces/api.py)
def get_cockpit(self):
    return _sentinel.cockpit(self.processor)
```

### Méthodes du pont (écrans Sentinel v2)

| Méthode                                   | Écran        | Description                    |
|-------------------------------------------|--------------|--------------------------------|
| <code>get_health()</code>                 | Health       | État des composants et modèles |
| <code>get_cockpit()</code>                | Cockpit      | KPI réels lus du MPI courant   |
| <code>ml_predict_format(p)</code>         | ML           | Classification format ATIH (to |
| <code>ml_predict_collision_risk(p)</code> | ML           | Score de risque de collision I |
| <code>ars_score_lot(p)</code>             | Sentinel ARS | Score qualité d'un lot avant e |
| <code>get_audit_events(n)</code>          | Audit        | Journal chaîné SHA-256 (art. 3 |
| <code>cespa_check()</code>                | CeSPA        | Règles réforme PSY juillet 202 |
| <code>get_heatmap_sectors()</code>        | Heatmap      | File active par code postal    |
| <code>workflow_pending(s, n)</code>       | Workflow     | Pipeline TIM, MIM, préflight,  |

### Sécurité du pont

Les appels restent à l'intérieur du processus : il n'existe ni port à protéger, ni jeton à gérer, ni CORS à configurer. Un autre processus du poste ne peut pas interroger le backend, car rien n'écoute. La surface d'attaque réseau est nulle par conception, et le test `tests/test_no_network.py` le garantit en intégration continue.

## Module ML - configuration et entraînement

### Modèles embarqués

| Modèle               | Algorithme    | Tâche                          | Métrique clé            |
|----------------------|---------------|--------------------------------|-------------------------|
| format_detector.json | XGBoost tuned | Classification format ATIH (58 | accuracy 0,77 - F1 0,70 |
| collision_risk.json  | XGBoost tuned | Scoring risque collision IDV   | AUC 1,00 - F1 1,00      |
| ddn_validity.pkl     | RandomForest  | Détection DDN suspecte         | AUC 0,86 - acc 0,99     |

### Entraînement

Les modèles sont entraînés sur un dataset 100 % synthétique fidèle aux spécifications ATIH 2000-2026 (58 variantes). Aucun fichier patient réel n'est utilisé (RGPD art. 9).

```
# Entraînement standard (50k samples)
python -m backend.ml.train

# Entraînement plus robuste (200k samples)
python -m backend.ml.train --samples 200000 --seed 42

# Réutiliser un parquet existant (gain ~10s)
python -m backend.ml.train --data-cache data/cache.parquet

# Résultat : backend/ml/models/*.json + *.pkl
#           + training_metadata.json (leaderboard + hashes SHA-256)
```

### Pipeline de benchmark

Pour chaque tâche, 4 algorithmes sont évalués en cross-validation stratifiée (5-fold) - XGBoost default, XGBoost tuned (Optuna, 30 trials), LightGBM, RandomForest. Le winner est sélectionné par AUC (binaire) ou F1 macro (multiclasse) et sérialisé. Le leaderboard complet est disponible dans training\_metadata.json.

### Ajouter un nouveau format ATIH

```
# 1. Editer le catalogue des specs
# backend/ml/synthetic.py :: ATIH_SPECS

ATIH_SPECS.append({
    'format': 'RPS',
    'version': 'P15',
    'year_from': 2027,
    'line_length': 166,
    'ipp_start': 20,
    'ipp_len': 20,
    'ddn_start': 40,
})
```

```
# 2. Relancer l'entraînement
```

```
python -m backend.ml.train --samples 200000
```

## Audit des modèles

training\_metadata.json contient pour chaque modèle - date d'entraînement, nombre de samples, seed, leaderboard complet des 4 candidats avec métriques test, hash SHA-256 du fichier modèle sérialisé. Ce fichier est versionné en Git pour tracabilité réglementaire.

## Sécurité et conformité RGPD

### Périmètre réseau

Aucun. L'application n'ouvre aucune socket et n'expose aucun service HTTP, même sur 127.0.0.1. La communication frontend/backend est in-process (pont pywebview). Le test `tests/test_no_network.py` échoue en CI si une couche serveur (Flask, FastAPI, uvicorn) est réintroduite.

### Validation des chemins

`os.path.abspath()` + vérification que le chemin résolu reste sous le dossier autorisé (path traversal guard). Côté C# - `SafePath.validate()` applique la même règle. Les liens symboliques pointant hors du dossier cible sont rejetés.

### Données patient (RGPD art. 9)

Les IPP et DDN sont traités en mémoire pendant le traitement de lot. Seule la base SQLite locale du profil utilisateur conserve le Master Patient Index. Aucune télémétrie, aucun envoi réseau, 100 % local.

### Audit log art. 30 RGPD

Chaque action sensible (résolution IDV, prédiction ML, scoring de lot, workflow) est journalisée dans une table SQLite append-only dont les entrées sont chaînées par SHA-256 (`backend/quality/audit.py`). L'intégrité de la chaîne est vérifiable via `audit_verify()`. Le journal est lisible par le DPO et l'ARS sur demande.

### Anonymisation exports recherche

Option k-anonymity  $k \geq 5$  disponible pour les exports destinés à la recherche (SNDS, Health Data Hub). Hash SHA-256 tronqué (12 chars) appliqué aux IPP dans les exports pseudonymisés. Conforme MR-007.

### Variables d'environnement et CI

```
# Variables d'environnement optionnelles (jamais committées)
SOVEREIGN_OPERATOR=<nom>      # opérateur inscrit dans l'audit
OLLAMA_BASE=<url>              # CimSuggester intranet (défaut: vide = off)
OLLAMA_MODEL=<modele>         # modèle Ollama (défaut: llama3.2:8b)

# CI scan sécurité (GitHub Actions)
bandit -r backend/ -ll         # vulnérabilités Python (bloquant)
pip-audit -r requirements.txt  # CVE dépendances
ruff check .                   # linting
```



## CI/CD - pipeline et qualité de code

### Pipeline GitHub Actions

Déclenchement - push sur main ou PR vers main. Environnement - ubuntu-latest, Python 3.12.

| Étape           | Commande                       | Bloquant     |
|-----------------|--------------------------------|--------------|
| Tests Python    | pytest tests/ -v --tb=short    | Oui          |
| Sécurité        | bandit -r backend/ -ll         | Oui          |
| Tests frontend  | node tests/frontend/test_activ | Oui          |
| CVE dépendances | pip-audit -r requirements.txt  | Non (veille) |
| Lint + format   | ruff check / ruff format --che | Non (veille) |

### Conventions de commit

Format - <type>(<scope>): <description>

Types - feat, fix, docs, refactor, test, chore.

Scope - backend, frontend, ml, guide, ci.

Langue - anglais impératif, minuscules.

Tirets longs (-) interdits dans les messages de commit.

Exemples valides -

feat(ml): add RPS P15 format to ATIH\_SPECS

fix(pmsi): prevent path traversal in export paths

### Pull Requests

Branche de base - main. Branche de travail - <type>/<description-courte>. Tests pytest verts obligatoires avant merge. Review par au moins 1 membre du projet. Squash merge - un seul commit par PR dans main.

### Release

Cycle trimestriel - V36 juillet 2026, V37 octobre 2026. Tag Git : vX.X.Y.Z (semver). GitHub Release auto-générée avec changelog depuis les commits. Binaire PyInstaller attaché à la release (Sovereign\_OS\_DIM.exe).

### Régénérer les guides PDF

```
# Guide métier (TIM, médecin DIM, chef de pôle)
```

```
python tools/generate_guide.py
```

```
# Guide développeur (DSI, contributeurs)
```

```
python tools/generate_guide_dev.py
```

```
# Mode d'emploi court (utilisateurs finaux)
```

```
python tools/generate_manual.py
```

## Performances et benchmarks

### Conditions de mesure

Poste de référence - Intel Core i5 8e gen, 16 Go RAM, SSD NVMe. Système : Windows 11 Pro. Python 3.12, xgboost 2.0, sqlite 3.45.

### Backend - traitement PMSI

| Opération                        | Mesure   | Notes                  |
|----------------------------------|----------|------------------------|
| Scan 1 000 fichiers (12 dossier) | 1,2 s    | Heuristique regex      |
| Identification 1 fichier         | 180 µs   | Regex + longueur ligne |
| Traitement lot 10 fichiers / 1   | 5 s      | 8 workers en pic       |
| Ecriture MPI SQLite 150k ligne   | 3 s      | Bottleneck principal   |
| Chargement MPI 500k lignes       | < 400 ms | Index sur ipp          |

### Module ML

| Opération                       | Mesure  | Notes                          |
|---------------------------------|---------|--------------------------------|
| Chargement 3 modèles au démarr  | 180 ms  | XGBoost JSON + RandomForest pk |
| format_detector (1 ligne)       | < 1 ms  | Inférence XGBoost              |
| collision_risk (1 IPP)          | < 1 ms  | Inférence XGBoost              |
| ddn_validity (1 ligne)          | < 1 ms  | Inférence RandomForest         |
| Batch 10 000 lignes (tous modè) | ~150 ms | Sans parallélisme              |

### Frontend (WebView2)

| Opération                      | Mesure   | Notes                        |
|--------------------------------|----------|------------------------------|
| Chargement initial dashboard   | < 400 ms | MPI 50k IPP                  |
| Rafraîchissement post-traiteme | < 200 ms | Recalcul agrégats en mémoire |
| Rendu donut Chart.js           | < 80 ms  | Canvas HTML5                 |
| Export PDF organigramme 3p     | 1,2 s    | fpdf2 vectoriel              |

Goulot principal - écriture SQLite pour les très gros lots (> 1 M lignes). Solution : traiter par trimestres successifs ou passer à WAL mode (journal\_mode=WAL).

## Contribuer au projet

### Dépôt

[https://github.com/Adam-Blf/sovereign\\_os\\_dim](https://github.com/Adam-Blf/sovereign_os_dim)

Licence - MIT (code) + LGPL (fpdf2).

Issues GitHub pour bugs et suggestions.

Contact direct - [adam.beloucif@psysudparis.fr](mailto:adam.beloucif@psysudparis.fr)

### Structure du dépôt

| Dossier / Fichier   | Contenu                        |
|---------------------|--------------------------------|
| backend/pmsi/       | Moteur ATIH : scan, MPI, expor |
| backend/orgchart/   | Structure Pôle/Secteur/UM, org |
| backend/quality/    | Audit chaîné SHA-256, workflow |
| backend/interfaces/ | Pont pywebview (api.py, _senti |
| backend/ml/         | Module ML : train.py, syntheti |
| frontend/           | HTML + Tailwind + Chart.js ven |
| tools/              | Générateurs PDF, moulinette FI |
| tests/              | Tests pytest (dont test_no_net |
| docs/               | PDF livrables, dossier DSI, re |
| main.py             | Entrée de l'application deskto |
| build.py            | Script PyInstaller (packaging  |

### Ajouter un format ATIH

1. Editer backend/ml/synthetic.py - ajouter l'entrée dans ATIH\_SPECS (format, version, year\_from, line\_length, positions IPP/DDN).
2. Relancer l'entraînement ML (python -m backend.ml.train --samples 200000).
3. Mettre à jour le détecteur heuristique dans backend/data\_processor.py (regex + longueur).
4. Ajouter un test dans tests/ pour le nouveau format.
5. Documenter dans docs/research/pmsi\_formats\_history.md.

### Tests

```
# Lancer tous les tests
pytest tests/ -v

# Tests d'un module spécifique
pytest tests/test_no_network.py -v

# Avec couverture
pytest tests/ --cov=backend --cov-report=html
```

Merci de contribuer ! Les contributions améliorant la couverture des formats ATIH ou la robustesse du ML sont particulièrement bienvenues.

## Références et ressources

### Documentation officielle ATIH

Notice technique 2026 -  
<https://www.atih.sante.fr/notice-technique-nouveautes-pmsi-mco-had-smr-psychiatrie-2026-0>  
Formats PMSI 2026 (Excel) - <https://www.atih.sante.fr/formats-pmsi-2026-0>  
Catalogue formats historiques - [format-pmsi.fr](https://www.atih.sante.fr/format-pmsi.fr)  
Plateforme DRUIDES - <https://www.epmsi.atih.sante.fr>

### RGPD et conformité

RGPD UE 2016/679 - art. 9 (données de santé), art. 30 (registre de traitement).  
HAS - Instruction DGOS/PF2 no 2019-116 (identitovigilance).  
ANS - Référentiel national d'identito-vigilance 2023.  
Health Data Hub - MR-007 (pseudonymisation SNDS).  
Instruction DGOS/PF2/2020/143 - rétention 5 ans.

### Dépôt et support

GitHub - [https://github.com/Adam-Blf/sovereign\\_os\\_dim](https://github.com/Adam-Blf/sovereign_os_dim)  
Issues - [https://github.com/Adam-Blf/sovereign\\_os\\_dim/issues](https://github.com/Adam-Blf/sovereign_os_dim/issues)  
Contact - [adam.beloucif@psysudparis.fr](mailto:adam.beloucif@psysudparis.fr)  
Alternance EFREI M1 Data Engineering - Fondation Vallée 2025-2027.

Ce guide développeur est généré automatiquement depuis `tools/generate_guide_dev.py` - mettre à jour le contenu dans ce fichier pour l'enrichir.